

The background features two abstract network graphs. One graph in the top right corner consists of green nodes connected by thin green lines. Another graph in the bottom left corner consists of blue nodes connected by thin blue lines. The nodes vary in size, with some being larger and more prominent than others.

WQFS

(World event & Quest Fusion System)

월드 이벤트 & 퀘스트
융합 시스템
(7주차)

202444086 이인희

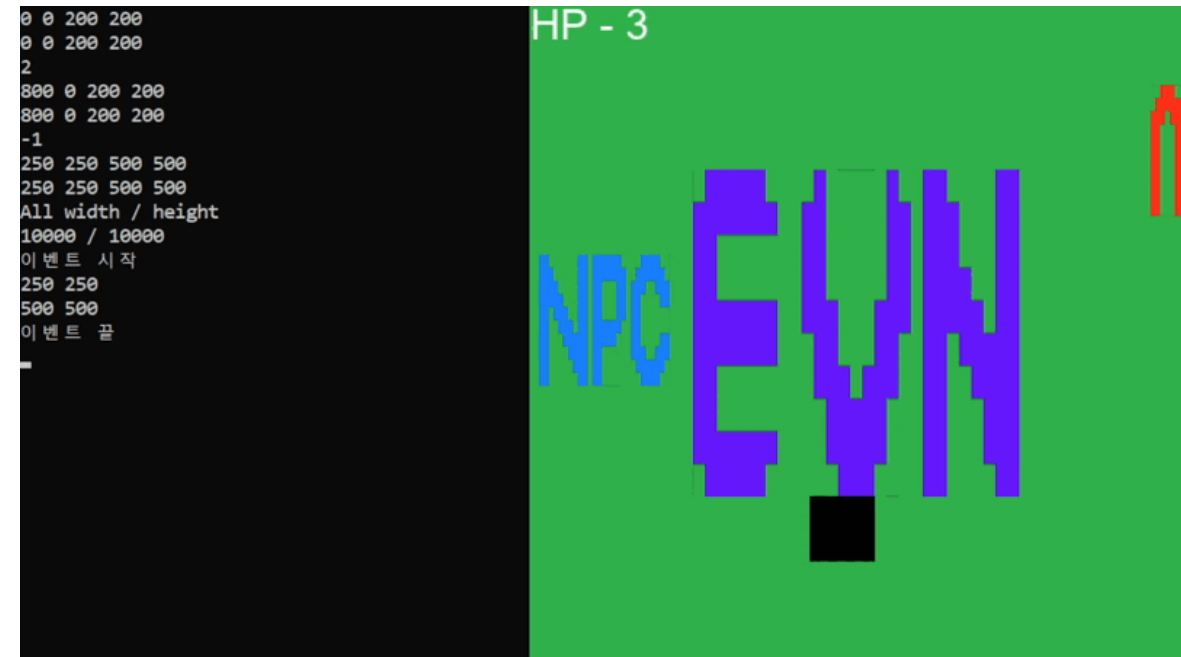
진행 상황

- WQFS

- 정적 기본 이벤트 구현
- 위치 고정, 긴 대기시간과 이벤트 지속 시간

```
void WQFS::CheckEvent() {  
    if (!WQFS::GetInstance().isPaused) {  
        for (auto& npc : WQFS::GetInstance().npcs) {  
            if (!npc.second.GetCanDangerous()) {  
                npc.second.Timer();  
            }  
        }  
        for (auto& event : WQFS::GetInstance().worldEvents) {  
            if (!event.second.GetIsCanCollid()) {  
                event.second.Timer();  
            }  
            else {  
                event.second.ResetTimer();  
            }  
        }  
    }  
}
```

번갈아가며 타이머 실행



0% 78% 100%

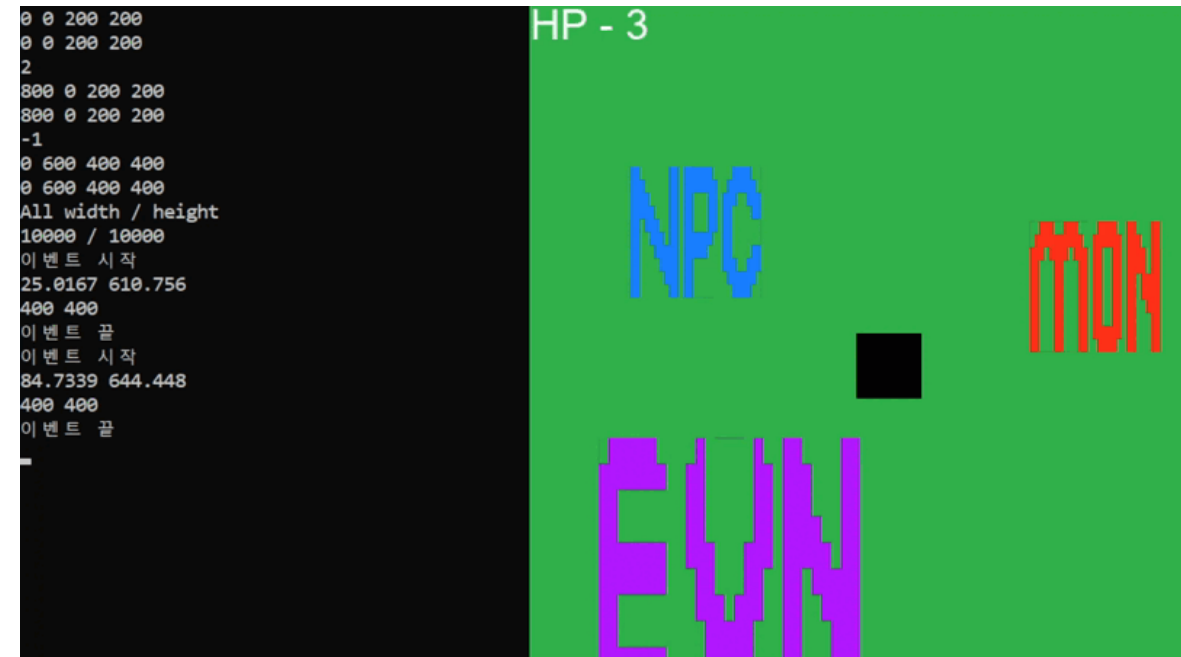
진행 상황

- WQFS

- 동적 기본 이벤트 구현
- 계속 위치 변경, 짧은 대기시간과 이벤트 지속 시간

```
void WQFS::CheckEvent() {  
    if (!WQFS::GetInstance().isPaused) {  
        for (auto& npc : WQFS::GetInstance().npcs) {  
            if (!npc.second.GetCanDangerous()) {  
                npc.second.Timer();  
            }  
        }  
  
        for (auto& event : WQFS::GetInstance().worldEvents) {  
            if (!event.second.GetIsCanCollid()) {  
                event.second.Timer();  
            }  
            else {  
                event.second.ResetTimer();  
            }  
        }  
    }  
}
```

번갈아가며 타이머 실행



0%  78%  100%

진행 상황

- WQFS 타이머

- 이벤트 지속시간 및 대기시간, 퀘스트 생성 딜레이 모니터링
- 각 객체는 각자 내부적으로 타이머를 가짐

```
void WorldEvent::Timer() {  
    if (maxTime == -1) {  
        return;  
    }  
  
    if (!doEvent && eventTimer > maxTime + pauseDuration) {  
        std::cout << "이벤트 시작" << std::endl;  
        eventStart = clock();  
        doEvent = true;  
    }  
  
    if (!isCanCollid && eventTimer > maxTime + collideDelay + pauseDuration) {  
        std::cout << "충돌 시작" << collideDelay << std::endl;  
        isCanCollid = true;  
    }  
  
    else if (!isCanCollid && eventTimer <= maxTime + collideDelay + pauseDuration) {  
        eventTimer = (double)(clock() - start) / CLOCKS_PER_SEC;  
    }  
}
```

타이머

```
void WorldEvent::ResetTimer() {  
    if (duration == -1) {  
        return;  
    }  
  
    if (durationTimer <= duration + pauseDuration) {  
        durationTimer = (double)(clock() - eventStart) / CLOCKS_PER_SEC;  
    }  
    else {  
        std::cout << "이벤트 끝" << std::endl;  
        start = clock();  
        eventTimer = 0.0f;  
        durationTimer = 0.0f;  
        pauseDuration = 0.0f;  
        doEvent = false;  
        isCanCollid = false;  
    }  
}
```

시간 리셋

```
WQFS::GetInstance().AddEvent("Landslide", 1, 2,  
WQFS::GetInstance().AddEvent("Earthquake", 2, 2,
```

...

```
andslide"].objSize.y, 3.0f, 3.0f, 0.3f);  
s["Earthquake"].objSize.y, 2.0f, 1.0f, 0.0f);
```

대기 시간, 지속 시간, 충돌 딜레이

0%

78%

100%

진행 상황

- WQFS 타이머

- 메뉴 창을 띄우는 등 시스템이 정지되었을 때 이벤트의 지속시간, 대기 시간 또한 정지

```
void WQFS::Pause() {
    WQFS::GetInstance().pauseStart = clock();
    WQFS::GetInstance().isPaused = true;
}

void WQFS::Resume() {
    double pauseDuration;
    pauseDuration = (double)(clock() - WQFS::GetInstance().pauseStart) / CLOCKS_PER_SEC;

    for (auto& npc : WQFS::GetInstance().npcs) {
        npc.second.SetPauseDuration(pauseDuration);
    }

    for (auto& event : WQFS::GetInstance().worldEvents) {
        event.second.SetPauseDuration(pauseDuration);
    }

    WQFS::GetInstance().isPaused = false;
}
```

```
if (glfwGetKey(window, GLFW_KEY_ESCAPE) && pauseDelayTimer > pauseDelay) {
    if (states == GAME_ACTIVE) {
        states = GAME_MENU;
        WQFS::GetInstance().Pause();
    }
    else if (states == GAME_MENU) {
        states = GAME_ACTIVE;
        WQFS::GetInstance().Resume();
    }

    pauseDelayTimer = 0.0f;
}
```



0%

78%

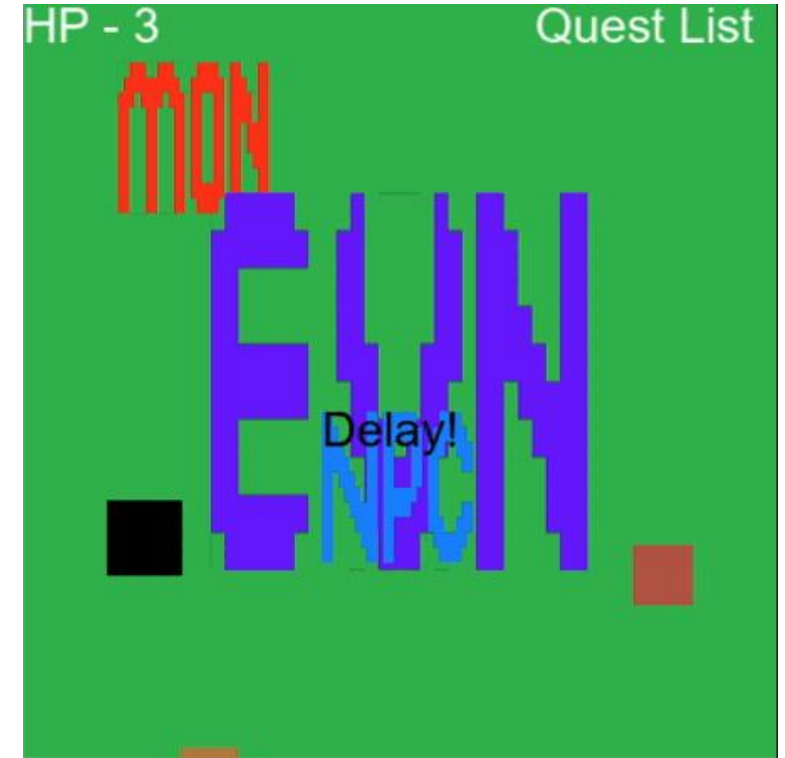
100%

진행 상황

• WQFS 이벤트

- 정적 이벤트와 충돌 딜레이를 이용하여 산사태 이벤트 구현
- 시스템은 객체와 충돌, 타이머, 퀘스트 생성 및 완료만 제공하므로 ParticleGenerator나 렌더링은 개발자가 따로 구현 필요

```
void ParticleGenerator::Update(float dt, GameObject& object, unsigned int newParticle, glm::vec2 offset, glm::vec2 direction) {  
    if (respawnDelay < 0.1f) {  
        respawnDelay += dt;  
    }  
  
    for (unsigned int i = 0; i < newParticle; ++i) {  
        if (respawnDelay >= 0.1f) {  
            int unusedParticle = FirstUnusedParticle();  
            RespawnParticle(particles[unusedParticle], object, offset, direction);  
            respawnDelay = 0;  
        }  
    }  
  
    for (unsigned int i = 0; i < amount; ++i) {  
        Particle& p = particles[i];  
        p.life -= dt;  
  
        if (p.life > 0.0f) {  
            p.Position += p.Velocity * dt;  
        }  
    }  
}
```



낙석 생성과 충돌시작 시간의 차이

0%

78%

100%

추후 일정

- 현재 구현된 기본 정적, 동적 이벤트를 기반으로 태풍, 쓰나미 등 다양한 이벤트를 구현할 수 있게 기능 추가 및 테스트 (시스템이 자체 충돌체크와 퀘스트 생성 및 완료를 지원하지만 낙석 생성 같은 랜더 요소이나 이동 등 요소는 개발자가 직접 구현해 제어)
- 현재 구현된 구출 퀘스트 외, 장애물 제거, 도둑 맞는 물건 되찾기 등 다양한 퀘스트 제작 및 상호작용한 이벤트에 따라 알맞은 퀘스트 생성 로직 구현
- 퀘스트 목록 같은 UI로 퀘스트를 표시할 수 있게 퀘스트 리스트를 문자열 형태로 반환하는 메소드 작성